

Arquitetura Inicial

Projeto	Formandos — gestão administrativa de formaturas e eventos
Marco do cronograma	22/09/2026 (Arquitetura inicial + organização do repositório Git)
Versão	1.0 — elaborada em 26/09/2026
Equipe	Luanderson Arlindo, Luiz Orlando, José Renato e Vinícius
Repositório	https://github.com/luandersonarlindo/Formandos

1. Visão geral

O Formandos é um **monólito web** em **Next.js 16 (App Router)** com **React 19** e **PostgreSQL**. Um único projeto contém as telas, a lógica do servidor e o acesso ao banco. A escolha privilegia **simplicidade**: uma equipe pequena, um só repositório, uma só linguagem (TypeScript) e nenhum serviço extra para manter.

```
Navegador (React 19)
  HTML do servidor + JavaScript (hidratação, Anime.js, Recharts)
    |
    v
Next.js 16 - servidor Node.js
|- src/proxy.ts ..... sem cookie de sessão? vai para /entrar
|- Páginas e layouts .... Server Components; checam acesso (dal.ts)
|- Server Actions ..... src/actions: validam (Zod), checam o
    |                               papel e gravam (SQL)
'- /api/auth/[...all] ... Better Auth (login, sessão, email, senha)
    |
    v
PostgreSQL (driver pg, SQL parametrizado) - 17 tabelas
|- Better Auth: usuarios, session, account, verification
'- Domínio: turmas, membros, catalogos, categorias, enquetes,
    opcoes, votos, duvidas, duvida_upvotes, tarefas, fornecedores,
    programacao, tentativas_convite

Serviços externos: Google OAuth (login) e SMTP/Gmail (emails)
```

2. Decisões de arquitetura

#	Decisão	Alternativas consideradas	Motivo
D1	Next.js App Router com Server Components e Server Actions	React puro + API separada; Pages Router	Menos código: a mesma aplicação renderiza, valida e grava, sem uma API REST paralela
D2	PostgreSQL com SQL puro (pg)	ORM (Prisma, Drizzle)	Aprender SQL de verdade; consultas simples e sob controle; sempre parametrizadas

#	Decisão	Alternativas consideradas	Motivo
D3	Better Auth	Auth.js (NextAuth v5, ainda beta), autenticação própria	Estável, guarda tudo no PostgreSQL, suporta Google e email e senha
D4	Uma tabela <code>membros</code> (usuário, turma, papel)	<code>turma_id</code> dentro de <code>usuarios</code>	O papel vale por turma ; permitir várias turmas para administradores foi só remover uma restrição
D5	URLs sem o id da turma	<code>/turmas/[id]/...</code>	O servidor descobre a turma pelo usuário logado e por um cookie de turma ativa, sempre conferido no banco
D6	Tailwind CSS 4 + shadcn/ui	CSS Modules, biblioteca de componentes fechada	Componentes acessíveis e copiados para o projeto; estilo rápido e consistente
D7	Zod para validar entradas	Validação manual	Uma fonte da verdade para formato e mensagens de erro
D8	Recharts para gráficos	Chart.js, SVG manual	Integra bem com React; gráficos de barras do relatório
D9	Sem hospedagem : roda localmente; código no GitHub	Vercel ou Render	Decisão do grupo; reduz custo e complexidade
D10	Anime.js para animações	CSS puro, Framer Motion	API simples; a animação é só um acréscimo, e o conteúdo aparece sem JavaScript

3. Camadas e organização do código

Camada	Pasta	Responsabilidade
Apresentação	<code>src/app</code> , <code>src/components</code>	Rotas, layouts, telas e componentes; <code>ui/</code> (genéricos) e <code>features/</code> (de domínio)
Aplicação	<code>src/actions</code>	Server Actions : as mutações (votar, criar, excluir...). Cada uma valida com Zod, confere o papel e executa o SQL
Acesso a dados	<code>src/lib/*.ts</code>	Consultas de leitura por área (<code>votacoes.ts</code> , <code>relatorio.ts</code> , <code>duvidas.ts</code> ...) e a conexão (<code>db.ts</code>)
Segurança	<code>src/lib/dal.ts</code> , <code>src/proxy.ts</code>	Verificações centralizadas: <code>exigirSessao</code> , <code>exigirMembro</code> , <code>exigirAdmin</code> , <code>exigirMaster</code>
Banco	<code>db/</code>	<code>schema.sql</code> , <code>apply-schema.mjs</code> e <code>seed.mjs</code> (catálogo padrão lido de <code>docs/catalogo-enquetes.md</code>)

Camada	Pasta	Responsabilidade
Testes	<code>src/lib/*.test.ts</code> , <code>scripts/teste/</code>	Testes unitários (Vitest) e testes de fluxo no navegador
Documentação	<code>docs/</code> , <code>README.md</code>	Guia de estudo, catálogo de enquetes e documentos de entrega

Grupos de rotas (pastas entre parênteses, que não aparecem na URL): `(publico)`, `(onboarding)`, `(app)`, `(admin)` e `(master)`. Cada grupo tem seu layout e sua regra de acesso.

4. Modelo de dados

Tabela	Conteúdo	Relações principais
<code>usuarios</code> , <code>session</code> , <code>account</code> , <code>verification</code>	Contas, sessões e tokens do Better Auth	<code>session.userId</code> e <code>account.userId</code> → <code>usuarios</code>
<code>turmas</code>	Nome, código de convite, data e local do evento	<code>criado_por</code> → <code>usuarios</code>
<code>membros</code>	Vínculo usuário × turma com o papel (<code>admin</code> ou <code>participante</code>)	chave (<code>turma_id</code> , <code>usuario_id</code>)
<code>catalogos</code>	Conjunto de enquetes; <code>turma_id</code> nulo = catálogo padrão global	→ <code>turmas</code>
<code>categorias</code> → <code>enquetes</code> → <code>opcoes</code>	Estrutura das votações; <code>enquetes.tipo</code> (única ou múltipla) e <code>opcoes.exclusiva</code>	encadeadas por chave estrangeira
<code>votos</code>	Voto identificado (opção × usuário), guardando a turma	→ <code>opcoes</code> , <code>usuarios</code> , <code>turmas</code>
<code>duvidas</code> , <code>duvida_upvotes</code>	Perguntas, resposta oficial, status e votos	→ <code>turmas</code> , <code>usuarios</code>
<code>tarefas</code>	Tarefas com responsável, prazo e status	→ <code>turmas</code> , <code>usuarios</code>
<code>fornecedores</code>	Vitrine de terceiros por turma	→ <code>turmas</code>
<code>programacao</code>	Itens de horário da festa	→ <code>turmas</code>
<code>tentativas_convite</code>	Códigos errados por usuário, para limitar tentativas	→ <code>usuarios</code>

Todas as tabelas do domínio apagam em cascata quando a turma ou o usuário é excluído. O esquema só usa `create table if not exists` e migrações pontuais com `if exists`, para poder ser reaplicado com segurança.

5. Fluxos principais

5.1. Autenticação e acesso

1. A pessoa entra por **Google** ou por **email e senha**; no cadastro por senha, o login só vale depois de **confirmar o email** pelo link.
2. O Better Auth grava a **sessão** no PostgreSQL e devolve um cookie.
3. O `proxy.ts` só confere se **existe** o cookie (checagem rápida). A **validação real** acontece em `lib/dal.ts`, chamada em toda página, layout e Server Action.
4. Sem turma, a pessoa vai a `/convite`; com turma, ao `/dashboard`. A turma em uso vem do cookie `turma_ativa`, **sempre conferido** contra as turmas de que ela é membro.

5.2. Uma ação de escrita (exemplo: votar)

1. O formulário envia os dados para a **Server Action** `votar`.
2. A action chama `exigirMembro()` (sessão e turma) e valida o corpo com **Zod**.
3. Numa **transação**, apaga os votos anteriores do usuário naquela pergunta e grava os novos.
4. A página é revalidada e a tela mostra o voto salvo.

6. Segurança em camadas

Camada	Proteção
Rede	Cabeçalhos HTTP (<code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code>) e <code>X-Powered-By</code> desligado
Acesso	<code>proxy.ts</code> (rápido) + <code>dal.ts</code> (definitivo) em páginas e ações; painel master responde 404 para quem não é master
Dados	Toda entrada validada com Zod; SQL sempre parametrizado; cascata nas exclusões
Abuso	Limite de 10 códigos de convite errados a cada 15 minutos por usuário; transação com trava para entrar em turma
Segredos	<code>.env.local</code> fora do Git; chaves lidas de variáveis de ambiente; <code>client_secret_*.json</code> no <code>.gitignore</code>
Privacidade	O relatório mostra só totais; quem votou só aparece para administradores

7. Configuração e ambientes

Variável	Uso
<code>DATABASE_URL</code>	Conexão com o PostgreSQL
<code>BETTER_AUTH_SECRET</code> e <code>BETTER_AUTH_URL</code>	Segredo das sessões e endereço do app
<code>GOOGLE_CLIENT_ID</code> e <code>GOOGLE_CLIENT_SECRET</code>	Login com Google (opcional: sem elas, o botão some)

Variável	Uso
SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASS , EMAIL_FROM	Envio de emails (sem SMTP_HOST , o email é escrito no terminal)
ADMIN_MASTER_EMAILS	Emails dos administradores master

Ambiente atual: **um só**, o de desenvolvimento local (Node.js e PostgreSQL na máquina). Não há hospedagem.

8. Qualidade

- **Tipos:** TypeScript estrito; `npm run typecheck` gera os tipos das rotas antes de conferir.
- **Testes unitários (Vitest):** regras puras de convite, datas, vínculos e master (20 testes).
- **Testes de fluxo no navegador:** roteiro em `scripts/teste/fluxos.mjs` (59 passos).
- **Auditoria de responsividade:** `scripts/teste/auditoria.mjs`.
- **Pontos ainda ausentes:** ESLint e integração contínua (previstos para a Sprint 2).

9. Riscos e dívidas técnicas

Item	Impacto	Plano
Next.js 16 muda APIs conhecidas (<code>proxy.ts</code> , <code>params</code> como <code>Promise</code> , <code>error.tsx</code> com <code>retry</code>)	Código de tutoriais antigos quebra	Ler a documentação em <code>node_modules/next/dist/docs/</code> antes de codar
SQL escrito à mão em muitos pontos	Repetição e risco de erro de digitação	Testes de fluxo cobrem os caminhos principais; extrair consultas comuns
Sem integração contínua	Erros só aparecem na máquina de quem roda	Adicionar <code>typecheck</code> e <code>npm test</code> a cada <code>push</code> na Sprint 2
Sem hospedagem	Não há teste em ambiente parecido com produção	Documentar o passo a passo local; avaliar hospedagem se o grupo decidir

10. Evolução prevista

Ligar a integração contínua, adicionar ESLint, ampliar os testes de ponta a ponta e, se houver tempo, tema escuro e edição de perguntas (ver *Sprint Backlog*).